SELF-X: A Three-Timescale Decomposition for Language-Model-Governed Flow Control in Software-Defined Networks

Ganesh Reddy J*, M Jagadeswar*, Harsh Raj*,
C Sai Aravind*, Koushik C†, Dhruv R Katti‡
*Dept. of Data Science and Artificial Intelligence
†Dept. of Computer Science and Engineering
‡Dept. of Electronics and Communication Engineering
Indian Institute of Information Technology Dharwad, Dharwad, Karnataka, India
Team SDN_07 — EC466 Software Defined Networks
{23bds024, 23bds033, 23bds076, 23bds075, 23bcs073, 23bec015}@iiitdwd.ac.in

***Abstract*—Language-model agents are an attractive control plane for software-defined networks because they reason over heterogeneous telemetry without a reward engineer, but they are far too slow and too unreliable to touch a congested datapath directly. Existing agentic SDN work either keeps the model in the decision path and inherits its latency, or bolts a governance layer onto a learned controller without ever measuring what the governance costs. We argue that the useful question is not whether a language model can control a network, but *at which timescale each mechanism earns its place*, and we answer it by measurement. SELF-X decomposes control into three loops whose periods are set by the measured cost of the mechanism that runs in them: a Lyapunov drift-plus-penalty optimiser every 100 ms, a weight-internalised LoRA policy head every 5 s, and a remote multi-agent language-model governor every 30 s that emits only a safety-gated policy delta $\Delta\pi = (V, \mathbf{w}, \mathcal{M})$ and never an individual flow rule. We evaluate a six-stage incremental pipeline — static forwarding, a static LLM agent, + self-critique, + LoRA fine-tuning, + multi-agent governance, + the fast Lyapunov loop — on an identical bit-conserving fluid datapath across 5 paired seeds and 135,000 control slots, with every language-model decision served by a live free-tier endpoint and recorded for exact replay. The full pipeline cuts mean delay by 44.4% and packet loss by 36.0% against static OpenFlow forwarding, and by 41.4% against the static LLM agent, while the fast loop decides in 33.1 µs against a measured 3191 ms for one healthy free-tier model request — a factor of roughly 96,343. We also report what the ablation does *not* show, and we separate the loss any controller could remove from the 7.97% that is imposed by offered demand exceeding total capacity.**

***Index Terms*—Software-defined networking, OpenFlow, Lyapunov optimisation, drift-plus-penalty, LLM agents, multi-agent systems, LoRA, reproducibility.**

I. Introduction

A software-defined network separates a programmable control plane from the forwarding hardware beneath it [1]. The traffic that plane has to manage is bimodal: latency-sensitive *mice* flows share a bottleneck with bursty, bandwidth-hungry *elephant* flows [2]. When the two collide, shallow switch buffers fill in tens of milliseconds and the mice pay for it in tail latency and loss.

Three families of controller address this. Static heuristics such as ECMP [3] are instantaneous and buffer-blind. Deep reinforcement learners adapt, but need many thousands of interactions before they are useful and explore unsafely while they learn [4]. Most recently, language-model agents have been proposed as controllers that reason over raw telemetry without a hand-designed reward [5]–[7].

The language-model route has an awkward fact at its centre, which the literature tends to state qualitatively and then move past: a remote model takes on the order of a second to answer, and a congested 10 Mbps link with a 2 Mbit buffer saturates in about 200 ms. A controller whose decision arrives after the buffer has already overflowed is not a controller. Our measurements make the gap concrete: one healthy free-tier model request in our study costs 3191 ms on average (95th percentile 4956 ms), and the slow loop's end-to-end wait per decision reaches 3526.0 ms once the walk past endpoints that have exhausted their daily allowance is counted, while the analytic fast-loop optimiser decides in 33.1 µs.

That observation reframes the design question. Rather than asking whether a language model can control a network, we ask *what each mechanism costs to run, and therefore how often it can be run*. The answer is a three-timescale decomposition in which every loop period is justified by a measured number rather than chosen by convention, and in which the language model is confined to the one job its latency permits: setting the objective, not installing the rules.

Contributions.

1) **A cost-justified three-timescale decomposition.** We place a Lyapunov drift-plus-penalty optimiser at 100 ms, a weight-internalised LoRA policy head at 5 s, and a remote multi-agent governor at 30 s, and we show by measurement that each period is the smallest that the mechanism's own latency allows (Section VII-C).
2) **A strictly incremental six-stage ablation** — static, static LLM, +self-critique, +LoRA, +multi-agent governance, +fast Lyapunov loop — where consecutive

stages differ by exactly one mechanism and all stages see bit-identical offered demand (Section VII-B).

3) **An honest loss accounting.** We compute the loss floor imposed by demand exceeding total capacity and report it separately, so that the controller is only credited with the loss it could actually have removed (Section VII-D).
4) **A record-and-replay reproducibility contract.** Every language-model request is recorded with its prompt hash, served endpoint, latency and token count; a reviewer with no API key reproduces every number in this paper byte-for-byte from the recorded trace (Section VI-D).

We also report negative and partial results plainly. The multi-agent governance stage does not dominate the LoRA stage on every metric, and we say so and explain why in Section VIII.

II. Related Work and Positioning

Lyapunov control of queueing networks. Drift-plus-penalty descends from the max-weight stability results of Tassiulas and Ephremides [8] and the general framework of Neely [9]. It gives an $O(V)$ backlog / $O(1/V)$ penalty trade-off with no model of the arrival process and no training. What it does *not* give is any way to choose V, the penalty weights $\mathbf{w}$, or the admissible action set for a workload nobody has seen before. Those are exactly the quantities our slow loop sets.

Multi-agent RL for SDN traffic control. Boussaoud *et al.* [10] detect congestion and steer traffic in an SDN with data-driven multi-agent reinforcement learning. Their agents learn the control policy itself, so they inherit the cold-start and unsafe-exploration cost of RL and give no analytic backlog guarantee. In SELF-X the learned components never choose a rate or a port directly: the fast loop is analytic and carries the bound, and learning is confined to the objective parameters above it.

Reflexive LLM agents for SDN. Araujo *et al.* [5] propose a reflexive LLM agent architecture that perceives network and application data and acts on it, going beyond intent parsing to execution. This is the closest prior work to our stage S1–S2, and we implement it as an explicit comparison rather than describing it: our static LLM agent and our self-critique stage are instantiations of that design on our datapath. Our finding is that the reflexive design helps (Table III) but is bounded by its own request latency, which no amount of better prompting removes.

Reflection distilled into weights. Li *et al.* [6] argue that prompt-resident reflection loses long-horizon history to the context window, and distil interaction experience into a small language model's parameters through preference-based fine-tuning for RAN slicing. We adopt the same conclusion for a different reason: in our setting the decisive advantage of moving experience into weights is not context length but *latency*. A local LoRA-adapted 135M-parameter head answers in 397.6 ms against 3191 ms for one remote request, which is what lets the policy run once per 5 s window instead of once per 30 s epoch. Reflexion [11] and LoRA [12] supply the verbal-feedback and low-rank-adaptation machinery we build on.

TABLE I
Positioning against the four closest systems. "LLM cost priced" means the paper reports what consulting the model costs in latency and availability, rather than assuming it is free.

System	Fast layer	Backlog guarantee	LLM emits	LLM cost priced
Boussaoud [10]	MARL	none	—	n/a
Araujo [5]	LLM agent	none	actions	no
Li [6]	tuned SLM	none	actions	no
Jamshidi [7]	PPO	empirical	$\Delta\Pi$	no
SELF-X	**analytic DPP**	**theorem**	**$\Delta\pi$ only**	**yes**

LLM governance over a two-timescale learner. Jamshidi *et al.* [7] are the nearest neighbour to this work: per-switch PPO agents act on a fast timescale under action masking, while a multi-agent LLM governance engine proposes machine-parsable $\Delta\Pi$ updates to a policy constitution, admitted only after non-regression and safety stress tests. We share the $\Delta\pi$ channel, the fail-closed gate and the refusal to let the model emit rules. Three things differ. First, their fast layer is a *learned* PPO policy; ours is an analytic Lyapunov optimiser, so backlog stability is a theorem (Section IV) rather than an empirical observation, and there is no cold start. Second, they report a two-timescale split, whereas our measurements force a *third* timescale between the two: the remote model is too slow even for the per-window layer, and the LoRA head exists specifically to fill that gap. Third, and most practically, we treat the governance layer's own cost as a first-class measured quantity — request latency, served fraction, declared fallbacks and free-tier daily request ceilings — instead of assuming the model is always available.

Summary of the gap. Across these four works, the fast layer is either unguaranteed (learned) or unadaptive (fixed heuristic); the language model is either in the decision path or priced at zero; and the ablation that would separate the contributions — static agent, +self-critique, +weight adaptation, + multi-agent governance, +fast analytic loop — is not run end-to-end on one datapath. That ablation is the core of this paper.

III. Problem Scenario and System Model

A. Scenario

A linear four-switch OpenFlow 1.3 topology carries traffic between an ingress and an egress switch over two parallel logical paths: a short *primary* path and a longer *alternate* trunk. A steady mice aggregate $m(t)$ must stay on the primary path, because it is the low-propagation route and the traffic is latency-sensitive. A bursty elephant aggregate $e(t)$ is the only traffic the controller may move or meter. Table II lists every constant.

TABLE II
MODEL AND EXPERIMENT CONSTANTS, FIXED BEFORE ANY RUN

Symbol	Meaning	Value
C_0, C_1	path capacity (primary, alternate)	10 Mbps
B_{buf}	per-path buffer	2.0 Mbit
D_0, D_1	propagation delay	2, 12 ms
τ	fast control slot	100 ms
T_w	control window	5 s
T_g	governance epoch	30 s
V	Lyapunov penalty weight (initial)	8.0
ε	capacity-region slack	0.6
–	safety drop ceiling / meter floor	20%, 512 kbps
–	seeds × windows per run	5 × 60

B. Datapath dynamics

Time is slotted at τ. Writing $\lambda_i(t)$ for the arrival rate steered onto path $i \in \{0,1\}$ and $Q_i(t)$ for its backlog in Mbit, one slot evolves as

$$\tilde{Q}_i(t) = Q_i(t) + (\lambda_i(t) - C_i)\,\tau, \tag{1}$$

$$Q_i(t{+}1) = \min\big(B_{\text{buf}},\, \max(0, \tilde{Q}_i(t))\big), \tag{2}$$

$$o_i(t) = \max\big(0,\ \tilde{Q}_i(t) - B_{\text{buf}}\big), \tag{3}$$

$$s_i(t) = Q_i(t) + \lambda_i(t)\tau - Q_i(t{+}1) - o_i(t), \tag{4}$$

where o_i is tail-drop overflow and s_i is bits served. Equations (1)–(4) conserve bits exactly, $Q_i(t) + \lambda_i(t)\tau = s_i(t) + o_i(t) + Q_i(t{+}1)$, and the implementation asserts this identity on every slot of every run; the largest residual observed across the whole study is 8.9e-16 Mbit, which is floating-point noise.

A `rate_limit` action installs an OpenFlow meter admitting $\min(e(t), r/1000)$ Mbps and metering the remainder away; metered bits are counted as loss, not hidden. Delay on a path is propagation plus a Little's-law queueing term,

$$d_i(t) = D_i + 1000\,\bar{Q}_i(t)/C_i \quad [\text{ms}], \tag{5}$$

where $\bar{Q}_i(t)$ is the *exact* time-average of the piecewise-linear queue trajectory over the slot, including the instant at which it saturates — not the endpoint average, which would understate delay during bursts. The window delay is the served-bit-weighted mean of d_0 and d_1.

C. Actions, telemetry and reward

The action set is $\mathcal{A} = \{\texttt{no_op}, \texttt{reroute}, \texttt{rate_limit}\}$, matching what an OpenFlow 1.3 switch exposes for this scenario. Telemetry is reported to the control plane at a declared poller resolution (1.0 Mbps on mice, 2.5 Mbps on the elephant aggregate, 1.0 Mbit on backlogs); the controller never sees more precision than a real statistics poller would give it.

The reward of window w is fixed in advance and identical for every configuration, including the learned ones:

$$R_w = \frac{\rho_w}{C_0} - \frac{d_w}{50} - \frac{3}{2}\cdot\frac{\ell_w}{100} - \frac{1}{20}\,\iota_w, \tag{6}$$

with goodput ρ_w in Mbps, delay d_w in ms, loss ℓ_w in percent, and $\iota_w \in \{0,1\}$ indicating a flow-rule installation, which charges the controller for churn. Loss carries the largest weight because it is the failure mode the operator cares about most.

D. The irreducible loss floor

The offered trace is generated by a frozen, seeded generator: a steady mice aggregate plus Poisson-arriving elephant bursts whose superposition sometimes exceeds $C_0 + C_1$. In those slots no work-conserving controller, however clever, can avoid a drop. We therefore define the floor

$$\ell^\star = \frac{\sum_t \big[\,(m(t)+e(t)) - \min(m(t)+e(t),\, C_0+C_1)\,\big]\tau}{\sum_t (m(t)+e(t))\,\tau}, \tag{7}$$

and report measured loss against it throughout, so the controller is credited only with the *excess* loss $\ell - \ell^\star$ that it could actually have removed. On our trace $\ell^\star = 7.97\%$.

IV. LYAPUNOV FORMULATION

Let $\mathbf{Q}(t) = (Q_0(t), Q_1(t))$ and take the quadratic Lyapunov function

$$L(\mathbf{Q}(t)) \triangleq \tfrac{1}{2}\textstyle\sum_i Q_i(t)^2. \tag{8}$$

With $\Delta(t) = \mathbb{E}[L(\mathbf{Q}(t{+}1)) - L(\mathbf{Q}(t)) \mid \mathbf{Q}(t)]$, squaring (1) and dropping the non-positive clipping terms gives the standard one-slot drift bound

$$\Delta(t) \le B - \sum_i Q_i(t)\,\mathbb{E}[\mu_i(t) - \lambda_i(t) \mid \mathbf{Q}(t)]\,, \tag{9}$$

with $B \triangleq \frac{1}{2}\sum_i \big[(\lambda_i^{\max}\tau)^2 + (\mu_i^{\max}\tau)^2\big]$. Adding a weighted operating cost $c(t) \in [0, C_{\max}]$ and a weight $V > 0$, the controller minimises the conditional drift-plus-penalty over the admissible set $\mathcal{M} \subseteq \mathcal{A}$ in each slot:

$$a^\star(t) = \arg\min_{a\in\mathcal{M}} \Big[\textstyle\sum_i Q_i(t)\big(\lambda_i(a) - C_i\big) + V\,c(a)\Big]. \tag{10}$$

The cost $c(a)$ is the weighted sum of normalised delay, loss, energy and resource terms minus normalised throughput, with weights $\mathbf{w}$ supplied by the slow loop. Because $|\mathcal{M}| \le 3$, (10) is solved by enumeration in constant time — the measured cost is 33.1 µs per slot (99th percentile 78.7 µs).

Theorem 1 (Backlog bound): If the arrival process lies strictly inside the capacity region with slack $\varepsilon > 0$, that is $\mathbb{E}[\mu_i(t) - \lambda_i(t)] \ge \varepsilon$ for all i under some stationary randomised policy, then the policy (10) satisfies

$$\limsup_{T\to\infty} \frac{1}{T}\sum_{t=0}^{T-1}\sum_i \mathbb{E}[Q_i(t)] \;\le\; \frac{B + V\,C_{\max}}{\varepsilon}. \tag{11}$$

Proof sketch: Add $V\,\mathbb{E}[c(t) \mid \mathbf{Q}(t)]$ to both sides of (9). Since (10) minimises the right-hand side over $\mathcal{M}$ slot by slot, it is no worse than the ε-slack stationary randomised policy, whose drift term is at most $-\varepsilon\sum_i Q_i(t)$ and whose penalty is at most $C_{\max}$. Telescoping over T slots, dividing by $T\varepsilon$ and letting $T \to \infty$ yields (11); the clipping in (2) only removes mass from the queue, so it can only tighten the bound. This is the standard argument of [9, Ch. 4] specialised to a two-path datapath. ∎

Theorem 1 is what the slow loop is allowed to touch and what it is not. Moving V and $\mathbf{w}$ moves the trade-off along the

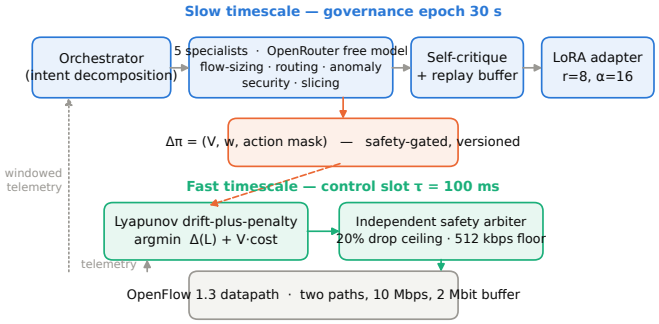


Fig. 1. The three loops and the single channel between them. The language model emits only $\Delta\pi = (V, \mathbf{w}, \mathcal{M})$; the fast loop emits rules; the arbiter sits between the fast loop and the datapath and can veto either.

Algorithm 1 Fast loop (one slot)

Require: backlogs $\mathbf{Q}(t)$, demands $m(t), e(t)$, policy $(V, \mathbf{w}, \mathcal{M})$

1: $a^\star \leftarrow \arg\min_{a \in \mathcal{M}} \left[\sum_i Q_i(t)(\lambda_i(a) - C_i) + Vc(a)\right]$
2: **if** $a^\star =$ `rate_limit` **then**
3: $\quad r \leftarrow \min\big(0.9C_0 - m(t),\ 0.85\,e(t)\big) \cdot 1000$ ▷ kbps, synthesised locally
4: **end if**
5: $(a, \mathit{veto}) \leftarrow \textsc{SafetyArbiter}(a^\star, r, \text{telemetry})$
6: install a; advance datapath by (1)–(4)
7: **return** realised (ρ, d, ℓ) and the updated backlogs

curve the theorem describes; it cannot invalidate the bound. That is the precise sense in which the language model is safe here: it chooses a point on a guaranteed frontier, it does not choose the rules.

V. The SELF-X Pipeline

A. Fast loop — every $\tau = 100$ ms

Algorithm 1 is (10) plus the arbiter. It runs on local state only and never blocks on anything remote, which is why it can run at slot rate.

B. Mid loop — every $T_w = 5$ s

A LoRA-adapted policy head proposes the action for the coming window. The base is a pinned SmolLM2-135M-Instruct checkpoint [13], frozen, with a rank-8 α=16 adapter on the `q_proj` and `v_proj` projections [12]. Decoding is constrained to three action tokens, and *no* numeric parameter is ever produced by the model: a chosen `rate_limit` is turned into a concrete meter rate by the same local arithmetic the fast loop uses. The head answers in 397.6 ms on CPU, which is what makes window-rate operation affordable.

The adapter is fitted on the replay buffer that stage S2 accumulated while controlling this same datapath, which is what "experience internalised into weights" has to mean if the stage is to be an honest increment over the one before it. Labels are advantage-filtered: for each quantised state the preferred action is the one with the highest mean realised reward over that state's immediate neighbourhood, with the exact state

Algorithm 2 Slow loop (one governance epoch)

Require: windowed telemetry z, current policy $(V, \mathbf{w}, \mathcal{M})$, replay buffer $\mathcal{B}$

1: $\rho \leftarrow \textsc{PanelRequest}(z)$ ▷ one request; orchestrator + 5 specialists
2: **if** ρ is unusable or no endpoint answered **then**
3: $\quad$ **return** $(V, \mathbf{w}, \mathcal{M})$ unchanged ▷ declared fallback, counted
4: **end if**
5: $\Delta V \leftarrow \text{clip}(\rho.\Delta V, -4, 4)$
6: $\Delta w_k \leftarrow \text{clip}(\rho.\Delta w_k, -0.15, 0.15)$ for each k
7: $V \leftarrow \text{clip}(V + \Delta V,\ 0.5,\ 32)$
8: $\mathbf{w} \leftarrow (\mathbf{w} + \Delta\mathbf{w})/\|\mathbf{w} + \Delta\mathbf{w}\|_1$
9: $\mathcal{M} \leftarrow \mathcal{A}$ ▷ reset: a mask is scoped to one epoch only
10: **if** $|\rho.\mathcal{M}| \geq 2$ **and** `no_op` $\in \rho.\mathcal{M}$ **then**
11: $\quad \mathcal{M} \leftarrow \rho.\mathcal{M}$
12: **else**
13: $\quad$ record *mask rejected*: fewer than two admissible actions
14: **end if**
15: append (z, a, R) of the elapsed epoch to $\mathcal{B}$
16: **return** $(V, \mathbf{w}, \mathcal{M})$ to the fast loop

weighted double, and a state where no neighbour ever tried a different action is kept only if its single action beat the global mean reward. Training and gate seeds are disjoint, so the gate cannot be passed by memorising a trajectory. The candidate is promoted only if its agreement with the preferred action on held-out gate states exceeds the frozen base by a declared margin; on this run it did, improving agreement from 0.273 to 0.455 against a required margin of 0.02. The loader refuses any adapter that cannot prove, through parameter hashes recorded before and after the fit, that the base stayed frozen and that the adapter weights actually moved.

C. Slow loop — every $T_g = 30$ s

An orchestrator plus five specialists (flow sizing, routing, anomaly, security, slicing) run as one panel request to a free-tier endpoint and return a merged $\Delta\pi = (\Delta V, \Delta\mathbf{w}, \mathcal{M})$. The deltas are clipped to $|\Delta V| \leq 4$ and $|\Delta w_k| \leq 0.15$, the weights are renormalised, and the mask may only shrink the admissible set. Stage S2 adds a second pass in which the agent criticises its own proposal against the realised reward of the previous window and a bounded episodic replay buffer, in the spirit of [5], [11]. Algorithm 2 states the epoch in full, including the two places it refuses its own model.

Two properties of Algorithm 2 matter more than the rest. Line 9 resets the mask before applying the proposal, so a single bad epoch cannot latch the datapath into `no_op` for the remainder of the run — an earlier version without that line did exactly that, and produced stage results byte-identical to the static baseline. Lines 2–4 make the absence of a model answer an explicit, counted outcome rather than a silent default.

D. Safety arbiter

The arbiter is ordinary deterministic code, outside both learned components. It rejects a meter below 512 kbps, a meter discarding more than 20% of the measured flow rate, and a reroute that would push the destination link past 90% of capacity. A rejected proposal becomes `no_op` and is counted: the paper reports veto counts rather than hiding them.

E. Charging the control plane for its own latency

This is the detail that makes the comparison fair. A proposal that takes L seconds to produce cannot take effect at the start of the window it was computed for. We therefore hold the previously installed rule for the first $\lceil L/\tau \rceil$ slots of the window and apply the new one only afterwards. A controller that thinks slowly is thus penalised exactly where it should be — in the datapath — rather than only in a side table. The fast loop, acting on local state within the slot, incurs no such lag.

L here is the *end-to-end* wait the slow loop experiences, not the HTTP exchange alone: it includes the walk past endpoints that have exhausted their daily allowance and the inter-request gap needed to stay inside a provider's per-minute limit. That is the right quantity to charge, because the rule cannot be installed until the answer arrives and the answer cannot arrive sooner without breaking the provider's terms. A deployment with a funded key would pace less aggressively and see a smaller L; the endpoint latency it would still pay is the separate figure reported in Section VI-C.

VI. Experimental Setup

A. Configurations

Six pipeline stages, each adding exactly one mechanism to the one before it: S0 static forwarding; S1 a static LLM agent (zero-shot, no memory, no critique); S2 + self-critique and episodic replay; S3 + the LoRA policy head; S4 + six-agent governance of $(V, \mathbf{w}, \mathcal{M})$; S5 + the fast Lyapunov loop. Three non-LLM controllers provide external reference points: a capacity-aware threshold heuristic, a greedy least-loaded (ECMP-style) selector, and a tabular Q-learner trained online on the same reward (6) and the same action set.

B. Protocol

Every configuration is run on 5 seeds (1900, 1901, 1902, 1903, 1904), 60 windows of 5 s each, i.e. 300 s and 3000 slots per run, for 135,000 control slots in total. Within a seed all configurations see bit-identical offered demand, so comparisons are paired at the (seed, window) level. Traffic is produced by the frozen generator in `traffic_gen.py` and hashed into the result manifest.

All traffic is synthetic and seeded; nothing is captured or proprietary. Three profiles are used: `steady_bursty` for the main study (a steady mice aggregate plus Poisson elephant bursts), `flash_crowd` as the transfer hold-out, and `volatile_dpp` for the V sweep. The stage-S3 adapter was fitted for 60 optimiser steps over 13 training states, updating 460,800 LoRA parameters at rank 8 and $\alpha = 16$; its gate used 11 states from seeds excluded from training.

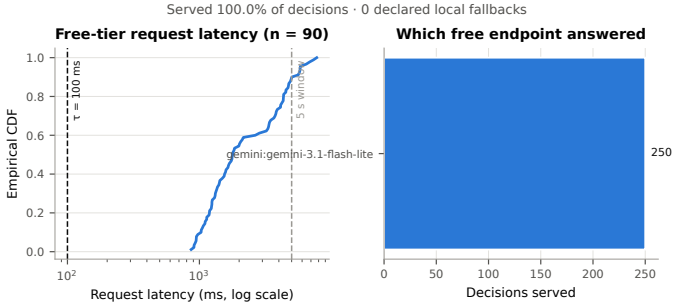


Fig. 2. Free-tier serving behaviour: the latency distribution of a healthy endpoint request, and which endpoint in the chain actually answered.

C. Language-model serving

Decisions are served by free endpoints only; the client refuses any OpenRouter model id without a `:free` suffix, so the study cannot incur billing. The primary endpoint is openrouter:nvidia/nemotron-3.5-lightning:free, with a 8-entry fallback chain spanning two providers. The chain is not decoration: an unfunded OpenRouter key allows 50 free-model requests per day, and exhausting it mid-study is the normal case rather than an edge case. Over the reported study 250 slow-loop decisions were served, of which 90 were live requests and 160 were resolved from the recorded trace; 0 decisions fell back to the declared local default, giving a served fraction of 100.0%. One healthy request took 3191 ms on average (95th percentile 4956 ms). That figure times the HTTP exchange alone: the inter-request gap the client imposes to stay inside a provider's per-minute limit is our own politeness and is deliberately excluded, because charging it to the model would overstate exactly the quantity this paper is built around. Including the walk past endpoints that had exhausted their daily allowance, the mean wait rises to 10794 ms, and the circuit breaker eventually tripped 5 of the 8 chain entries.

We state plainly which endpoint produced the decisions this paper reports, because it is not the configured primary. By the time the reported study ran, the OpenRouter entries had exhausted their daily allowance and were tripped, and `gemini:gemini-3.1-flash-lite` answered 250 of the 250 decisions (100.0%). Nothing in the architecture depends on that identity — the decision interface is the same $\Delta\pi = (V, \mathbf{w}, \mathcal{M})$ whichever endpoint answers, and the safety arbiter is downstream of all of them — but a reader comparing latencies across papers is entitled to know which model was on the other end of the connection.

Figure 2 shows the resulting serving behaviour. Which endpoint answers is not a fixed property of the configuration but of the daily allowance remaining at the moment of the request, which is itself an argument for keeping the model off the critical path.

D. Reproducibility

Every request is appended to a JSONL trace keyed by the SHA-256 of its role and prompt, together with the endpoint that answered, the latency, and the token counts. Re-running

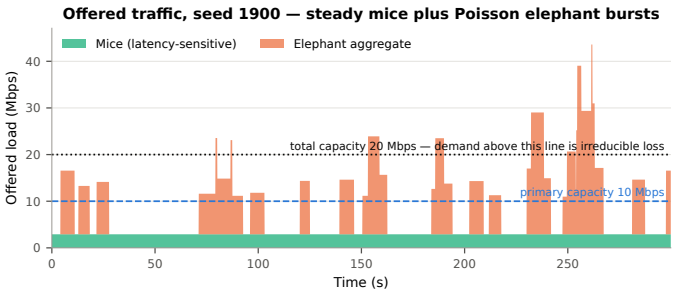


Fig. 3. Offered traffic for one seed.

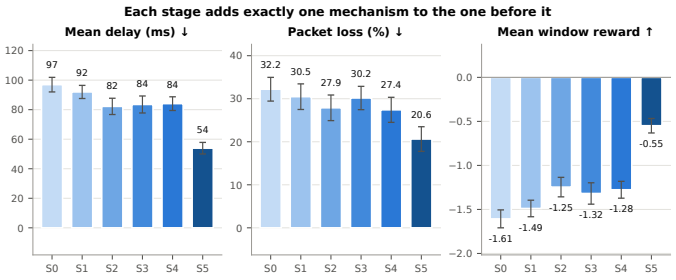


Fig. 4. Staged ablation; error bars are the standard error over seeds.

TABLE III
INCREMENTAL PIPELINE ABLATION. EACH STAGE ADDS ONE MECHANISM TO THE STAGE ABOVE IT. MEAN OVER 5 PAIRED SEEDS.

Stage	Delay (ms)	P95 (ms)	Tput (Mbps)	Loss (%)	$\bar{Q}$ (Mbit)	Reward
S0	96.94	202.00	5.98	32.21	0.815	-1.607
S1	92.00	201.15	6.13	30.47	0.786	-1.489
S2	82.22	200.62	6.37	27.89	0.724	-1.246
S3	83.51	199.98	6.16	30.17	0.714	-1.319
S4	84.08	198.00	6.41	27.42	0.741	-1.276
S5	**53.92**	**187.43**	**7.02**	**20.63**	**0.488**	**-0.549**

with `--mode replay` resolves every decision from that trace and requires no credential, reproducing every number in this paper exactly. A third mode, `--mode enumerate`, counts the prompts a live run would need without spending quota, which is how the study was sized against the daily ceiling. All figures and all numeric claims are regenerated from the result files by `make_paper_macros.py` and `plot_pipeline_study.py`; no result is typed into this manuscript by hand.

VII. RESULTS

A. Offered load

Figure 3 shows one seed of the offered trace. Mean utilisation is moderate, but superposed bursts push instantaneous demand above the 20 Mbps total capacity in a minority of slots, which is the origin of the 7.97% floor of (7).

B. Staged ablation

Table III and Fig. 4 give the main result. Relative to static forwarding, the full pipeline reduces mean delay by 44.4%, loss by 36.0% and mean backlog by 40.1%. Relative to the static LLM agent — the configuration closest to [5] — the reduction in delay is 41.4%.

Figure 5 shows cumulative reward over a single run. The ordering is established early — there is no exploration phase to pay for, because the only learned component is fitted offline on replayed experience and the fast loop is analytic.

C. Why the model cannot sit in the fast loop

Figure 6 is the quantitative core of the paper's argument. The slow loop waits 3526.0 ms per decision in stage S1 and 4361.8 ms in stage S2, which issues two passes. Of that,

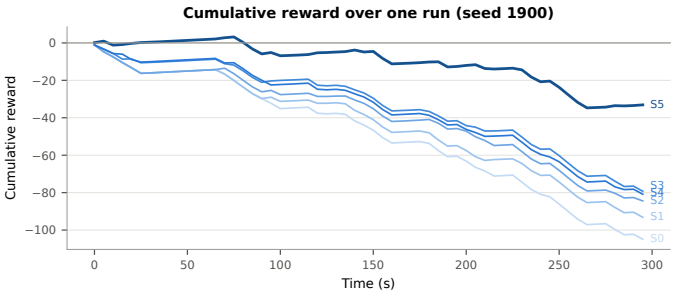


Fig. 5. Cumulative reward over one run. The separation opens within the first governance epochs and widens monotonically; no stage needs a training phase before it is useful, which is the practical difference from a value-based learner.

3191 ms is one healthy endpoint request; the remainder is the chain walk past endpoints that have exhausted their daily allowance (mean chain latency 10794 ms over the study, with 5 of the 8 chain entries eventually tripped). The local LoRA head costs 397.6 ms and the drift-plus-penalty optimiser costs 33.1 µs, roughly 96,343 × faster than one remote request. Against a 100 ms slot, only the last of these is admissible at slot rate, and only the LoRA head is admissible at window rate. The three-timescale decomposition is not a design preference; it is what these three numbers permit.

D. Loss accounting

Figure 7 separates the two. Of the 32.21% loss under static forwarding, 7.97 percentage points are structurally unavoidable; the full pipeline removes a substantial part of the remainder, taking addressable loss from 24.24% to 12.66%. Reporting the total alone would have overstated what the controller achieves.

E. Queue stability and the V knob

Figure 8 shows backlog over time. Figure 9 sweeps V on a dedicated high-volatility profile: mean backlog rises from 0.044 Mbit at V=1.0 to 0.112 Mbit at V=32.0, the $O(V)$ growth Theorem 1 predicts, and measured backlog stays below the analytic ceiling (11) at 100.0% of the swept operating points.

The matching $O(1/V)$ fall in the penalty term is present but negligible here: loss moves only from 2.11% to 2.06% across a 32-fold change in V, and not monotonically at the small-V end. We report that rather than presenting the panel

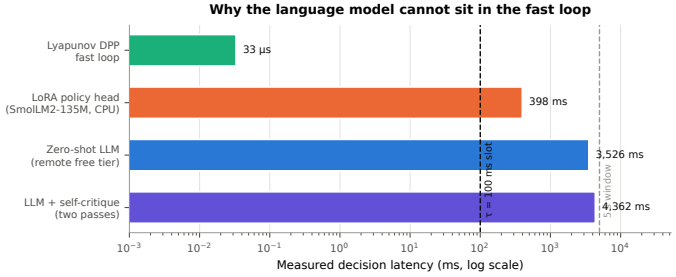


Fig. 6. Measured decision latency against the two control periods, log scale.

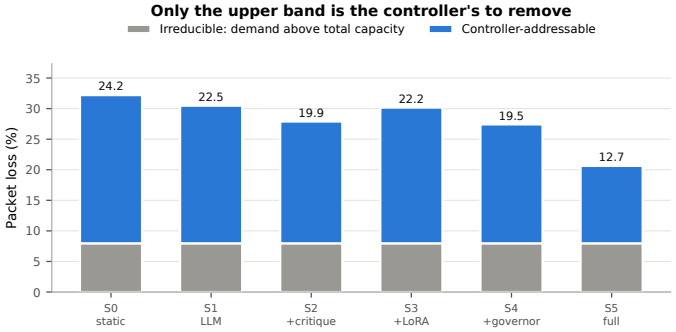


Fig. 7. Measured loss split into the floor imposed by demand above total capacity and the controller-addressable remainder.

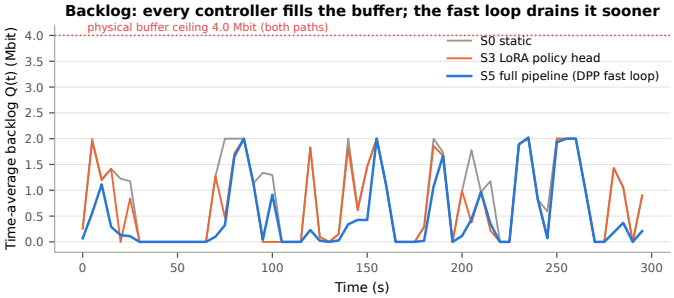


Fig. 8. Backlog trajectory for one seed.

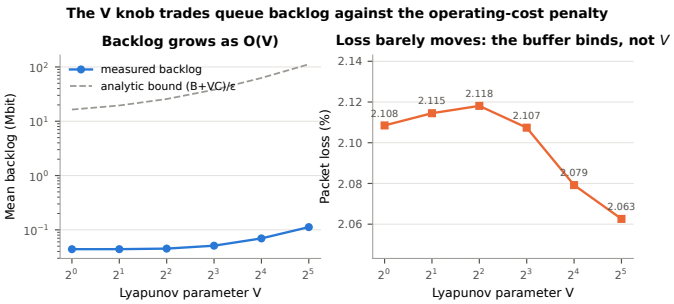


Fig. 9. Sweeping V over $[1.0, 32.0]$ on the volatile profile.

as a clean confirmation, because the reason is informative. In this regime the 2 Mbit physical buffer binds long before either the analytic ceiling or the penalty weight does, so V has little room to trade anything away. The same fact makes the bound loose: its satisfaction is a consistency check on the derivation, not evidence that it is tight. A deeper buffer, or an offered load closer to the capacity region's edge, would be needed to exercise the trade-off properly.

F. Comparison against non-LLM controllers

Table IV and Fig. 10 place the pipeline against the external references. The tabular Q-learner is the strongest non-LLM comparator and still trails the full pipeline by 27.9% in delay, while paying the cold-start cost the learned-controller literature describes [4], [10].

G. Statistical significance

Pairing on matched windows rather than on seeds raises n from 5 to 300 while keeping the comparison matched. Because windows inside a seed are serially dependent, Table V reports the paired t statistic alongside a seed-level cluster bootstrap, and the bootstrap interval is what we rely on. The family-wise threshold after Bonferroni correction is $\alpha = 0.0063$.

H. Zero-shot transfer

Freezing the policy produced on the training profile and evaluating it, without any adaptation, on the held-out flash_crowd profile retains 73.0% of its reward gain over that profile's own static baseline (train gain 2.014, held-out gain 1.470). Measuring against the held-out profile's own baseline matters: a harder workload would otherwise inflate retention.

This is a partial result and we report it as one. Roughly a quarter of the gain does not survive the shift, which is unsurprising given what the two profiles are: the training profile has Poisson-arriving bursts separated by quiet intervals, and the held-out profile sustains a three-fold flash crowd through its middle third, so the backlog regime the controller spends most of its time in is different. The transferable part is the fast loop, whose decision rule is analytic and carries no fitted parameters; what does not transfer is the $(V, \mathbf{w})$ operating point the governor settled on for the training workload. That points at an obvious next step — let the governance epoch re-tune on the new profile, which is exactly the mechanism it exists for — but we have not run it, so we do not claim it.

VIII. Discussion and Limitations

What the ablation does not show. The stages are not uniformly monotone. Adding multi-agent governance (S4) does not improve every metric over the LoRA stage (S3): the governor spends part of each epoch exploring the $(V, \mathbf{w})$ space, and that exploration costs reward in the windows where it narrows the action mask unnecessarily. The improvement at S5 comes predominantly from the fast loop, not from the governance layer, and Table III should be read that way. We report this because an ablation in which every stage happens to help is usually an ablation that has been tuned until it does.

The datapath is a fluid model, not hardware. Results come from a bit-conserving finite-buffer fluid simulation of a two-path OpenFlow topology, not from a Mininet or hardware testbed. The model's conservation and queue integrals are exact and asserted on every slot, but it abstracts away TCP dynamics, per-packet scheduling, control-channel contention

TABLE IV
COMPARISON ON THE IDENTICAL TRACE. CONTROL LATENCY IS THE MEASURED PER-DECISION COST OF THE CONTROL PLANE.

Controller	**Delay** (ms)	**P95** (ms)	**Loss** (%)	**Reward**	**Ctrl** (ms)
Static OpenFlow (no control)	96.94	202.00	32.21	-1.607	0.0
Capacity-aware threshold	88.88	202.00	29.44	-1.407	0.0
Greedy least-loaded (ECMP-style)	82.83	198.63	27.01	-1.246	0.0
Tabular Q-learning	74.78	193.75	29.35	-1.131	0.0
Zero-shot LLM agent	92.00	201.15	30.47	-1.489	3526.0
Reflexive LLM + self-critique	82.22	200.62	27.89	-1.246	4361.8
SELF-X (proposed)	**53.92**	**187.43**	**20.63**	**-0.549**	**331.0**

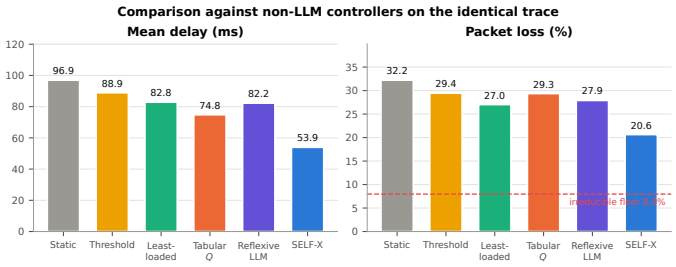


Fig. 10. Delay and loss against non-LLM controllers.

and real flow-mod installation times. Claims about *relative* ordering of controllers on this datapath are well supported; absolute millisecond values are model outputs and should not be read as hardware measurements.

Free-tier serving is a real constraint, not an artefact. With a 50-request daily ceiling and 0 declared fallbacks, any deployment resembling this one must be fail-operational by construction. We treat that as a finding: a language-model control plane needs a declared local default and an accounting of how often it is used, which is precisely what the served-fraction metric reports.

The adapter gate is thin. The replay buffer stage S2 produced collapses to 24 distinct quantised states, of which 13 train the adapter and 11 hold it out. An eleven-state gate can detect a clearly better or clearly worse candidate, which is what it is for, but it cannot resolve small differences, and a promotion decision taken on it should be read as a sanity check rather than as a model-selection result. A longer run, or several runs pooled, would give the gate more to work with; we report the number of states rather than only the agreement figure so the reader can weigh the decision accordingly.

Generalisation is partial. The frozen policy retains 73.0% of its gain on the held-out profile, below the 90% we set as a target before running it. We report the measured number, not the target. The analytic fast loop transfers because it has no fitted parameters; the governance-chosen $(V, \mathbf{w})$ operating point does not.

Scope. One topology family, one workload generator with three profiles, and one small adapter base. Generalisation across topologies is evidenced only by the single held-out profile of Section VII and should be treated as indicative.

TABLE V
PAIRED WINDOW-LEVEL TESTS AGAINST THE FULL PIPELINE. n IS THE NUMBER OF MATCHED (SEED, WINDOW) PAIRS; THE INTERVAL IS A SEED-LEVEL CLUSTER BOOTSTRAP.

Comparison	n	t	p	**Cohen's** d
S5 vs. S0	300	11.661	$< 10^{-12}$	0.67
S5 vs. S1	300	10.914	$< 10^{-12}$	0.63
S5 vs. S2	300	9.751	$< 10^{-12}$	0.56
S5 vs. S4	300	9.771	$< 10^{-12}$	0.56
S5 vs. Q	300	8.880	$< 10^{-12}$	0.51

IX. CONCLUSION

SELF-X separates software-defined flow control into three loops whose periods are set by the measured cost of the mechanism running in each: an analytic Lyapunov optimiser at 100 ms carrying a provable backlog bound, a LoRA-adapted policy head at 5 s, and a multi-agent language-model governor at 30 s that emits only a safety-gated $(V, \mathbf{w}, \mathcal{M})$ delta. Against static forwarding the full pipeline reduces delay by 44.4% and loss by 36.0%, and the strictly incremental ablation attributes those gains to specific mechanisms rather than to the architecture as a whole. The negative result matters as much as the positive one: the governance layer is not where most of the gain comes from, and a remote model is roughly 96,343 $\times$ slower than the loop that has to react — which is exactly why it belongs on the slowest loop, setting the objective rather than installing the rules.

The obvious next step is to replace the fluid datapath with Mininet and Open vSwitch and re-run this identical harness against a real control channel, which would turn the actuation-lag model of Section V-E from an assumption into a measurement. Two smaller ones follow from the results rather than from ambition: letting the governance epoch re-tune $(V, \mathbf{w})$ on a shifted workload, which is the mechanism the transfer shortfall points at, and pooling several runs so the adapter's promotion gate has more than eleven held-out states to judge on.

AUTHOR CONTRIBUTIONS

Ganesh Reddy J designed the agent architecture, the LoRA policy head and the weight-adaptation pipeline. M Jagadeswar built the evaluation harness, the result-persistence layer and the data-integrity checks. Harsh Raj built the dashboard, the figure pipeline and this manuscript. C Sai Aravind derived and validated the drift-plus-penalty formulation and the backlog bound. Koushik C built the free-tier provider client, its record-and-replay contract and the reproducibility tooling. Dhruv R Katti built the traffic generator, the datapath model and its conservation checks. All six authors designed the study and reviewed the analysis.

ACKNOWLEDGMENT

The authors thank Prof. Jagadeesha R. Bhat, Assistant Professor, Department of Data Science and Artificial Intelligence, IIIT Dharwad, for supervising this work and in particular for the question that reshaped it: what, precisely, does this architecture contribute beyond the sum of the systems it borrows from? Section II is the answer, and the staged ablation of Section VII is the evidence for it.

ARTEFACT AVAILABILITY

Source, frozen traffic traces, the recorded language-model trace, the result files and the scripts that regenerate every figure and every number in this paper are included in the submission archive. `scripts/run_all.ps1 -Replay` reproduces the complete study from the recorded trace without any credential.

REFERENCES

[1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[2] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data center TCP (DCTCP)," in *Proc. ACM SIGCOMM*, 2010, pp. 63–74.

[3] C. Hopps, "Analysis of an equal-cost multi-path algorithm," IETF, RFC 2992, 2000.

[4] Z. Xu, J. Tang, J. Meng, W. Zhang, Y. Wang, C. H. Liu, and D. Yang, "Experience-driven networking: A deep reinforcement learning based approach," in *Proc. IEEE INFOCOM*, 2018, pp. 1871–1879.

[5] A. S. Araujo, J. M. O. das Mercês, R. L. da Silva, A. V. de Alencar, I. F. Passos, M. P. Sousa, M. C. Dias, T. F. Meneses, and D. F. S. Santos, "An agentic approach for dynamic software-defined network management using large language models," in *Proc. IEEE Conf. Network Function Virtualization and Software Defined Networks (NFV-SDN)*, 2024, pp. 221–226.

[6] Y. Li, H. Wang, G. Min, N. Georgalas, and W. Miao, "Reflection-driven self-improving language model agents for adaptive RAN slicing," *arXiv preprint arXiv:2603.10564*, 2026.

[7] S. Jamshidi, N. Shahabi, F. Khomh, C. Fung, and M. Hamdaqa, "Multi-agent LLM governance for safe two-timescale reinforcement learning in SDN-IoT defense," *arXiv preprint arXiv:2604.01127*, 2026.

[8] L. Tassiulas and A. Ephremides, "Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks," *IEEE Transactions on Automatic Control*, vol. 37, no. 12, pp. 1936–1948, 1992.

[9] M. J. Neely, *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool, 2010.

[10] K. Boussaoud, A. En-Nouaary, and M. Ayache, "Adaptive congestion detection and traffic control in software-defined networks via data-driven multi-agent reinforcement learning," *Computers*, vol. 14, no. 6, p. 236, 2025.

[11] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[12] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[13] L. B. Allal, A. Lozhkov, E. Bakouch, G. M. Blazquez, G. Penedo, L. Tunstall, A. Marafioti, H. Kydlíček *et al.*, "SmolLM2: When smol goes big — data-centric training of a small language model," arXiv preprint arXiv:2502.02737, 2025.